CURSO 2026 - 2027

DOCUMENTO DE RESUMEN

UT01 · Resumen
Fundamentos de Bases de Datos NoSQL y MongoDB

CICLO FORMATIVO

Desarrollo de Aplicaciones Web (DAW) · 2º curso

MÓDULO

Tecnologías Web Actuales (Optativa)

UNIDAD DE TRABAJO

UT01 · Fundamentos MongoDB

Miguel Ángel Rubiano Barros
I.E.S. Suárez de Figueroa — Departamento de Informática y Comunicaciones

Bloque 1 · NoSQL y MongoDB

1. ¿Qué son las bases de datos NoSQL?

NoSQL = "Not Only SQL" (no solo SQL). Agrupa sistemas que no siguen el modelo relacional de tablas/filas/columnas con esquema fijo. Surgen para manejar grandes volúmenes de datos no estructurados, escalar horizontalmente (añadir servidores, no un servidor más potente) y ofrecer esquema flexible.

> **Idea clave.** NoSQL no es una tecnología concreta, sino una **categoría** que agrupa enfoques muy distintos. MongoDB es solo uno de los representantes, de la familia **documental**.

Las 4 familias NoSQL

Modelo	Unidad de almacenamiento	Ejemplos	Caso de uso típico
Documental	Documento JSON/BSON	MongoDB, Firebase	Catálogos, contenido web, APIs
Clave-valor	Par clave → valor	Redis, DynamoDB, Riak	Caché, sesiones, colas
Columnar	Columnas de una tabla ancha	Cassandra, Bigtable	Analítica sobre grandes volúmenes
Grafos	Nodos y aristas	Neo4j, OrientDB	Redes sociales, recomendaciones

SQL vs NoSQL

SQL	NoSQL
Esquema fijo y predefinido	Esquema flexible y dinámico
Modelo relacional (tablas)	Modelos variados (documentos, grafos, clave-valor, columnas)
Escalabilidad vertical	Escalabilidad horizontal
Propiedades ACID	Modelo BASE (Basically Available, Soft state, Eventual consistency)
SQL estandarizado	APIs específicas de cada motor

Ninguno sustituye al otro: la elección depende del problema. Un sistema bancario sigue encajando mejor en relacional; un catálogo de productos (como el de nuestra tienda_online) encaja de forma natural con el modelo documental.

2. MongoDB en concreto

Característica	Valor
Tipo	Base de datos documental
Lanzamiento	2009
Lenguaje de implementación	C++
Formato de almacenamiento	BSON (Binary JSON)
Licencia	Código abierto (SSPL desde la v4.4)

Jerarquía: base de datos → colección → documento

- **Documento**: unidad básica de almacenamiento, en BSON, sin esquema fijo obligatorio (equivalente a una fila SQL).
- **Colección**: grupo de documentos, normalmente con estructura similar (equivalente aproximado a una tabla SQL).
- **Base de datos**: contenedor de colecciones, con su propio namespace.

`db.productos.find()` se lee: "en la base de datos activa (`db`), dentro de la colección `productos`, busca documentos".

Características técnicas clave

- **CRUD** completo, framework de **agregación**, **índices secundarios**.
- **Réplica sets** (alta disponibilidad) y **sharding** (partición horizontal automática).
- Autenticación, autorización por roles, encriptación en tránsito y en reposo.

Ecosistema de herramientas

Herramienta	Qué es
MongoDB Atlas	Servicio de base de datos en la nube (DBaaS), con gestión automática de infraestructura
MongoDB Compass	Interfaz gráfica oficial para explorar datos y construir consultas visualmente
mongosh	Shell interactivo de línea de comandos (el que usamos en el curso)
MongoDB Charts	Visualización de datos y dashboards
Drivers oficiales	Bibliotecas cliente: Node.js, Python, Java, C#, PHP, Ruby...

Bloque 2 · Instalación, mongosh y BSON

3. mongosh: comandos básicos

Para trabajar en local hacen falta dos piezas: **MongoDB Community Server** (el motor, proceso `mongod`) y **mongosh** (el shell con el que escribimos comandos). Compass es la interfaz gráfica opcional de apoyo.

Comando	Qué hace
show dbs	Lista todas las bases de datos existentes en el servidor
use nombre_bd	Cambia (o crea, si no existe) la base de datos activa
db	Muestra el nombre de la base de datos activa
show collections	Lista las colecciones de la base de datos activa
db.help()	Muestra la ayuda de comandos disponibles sobre db
exit	Cierra la sesión del shell

Idea clave. mongosh es, en realidad, un entorno de **JavaScript**: además de los comandos propios de MongoDB puedes usar variables, bucles y funciones estándar.

4. BSON: los tipos de datos de MongoDB

MongoDB no guarda JSON como texto: lo convierte a **BSON** (Binary JSON), más eficiente y con más tipos de datos que el JSON plano (fechas reales, `ObjectId`, decimales de alta precisión...).

Tipo BSON	Ejemplo	Equivalente SQL
String	"Smartphone Pro"	VARCHAR / TEXT
Number (Int32/Int64)	25	INT / BIGINT
Double	699.99	FLOAT / DOUBLE
Decimal128	NumberDecimal("699.99")	DECIMAL (alta precisión)
Boolean	true / false	BOOLEAN
Date	ISODate("2026-10-07")	DATE / DATETIME
Array	["a", "b", "c"]	Sin equivalente directo
Object / embebido	{ calle: "..." }	Sin equivalente directo
ObjectId	ObjectId("6ab6d168...")	Clave primaria autogenerada

Atención. Todo documento tiene un campo `_id` que actúa como **clave primaria** y es, además, el único campo sobre el que **siempre** existe un índice. Si no lo indicas, MongoDB genera automáticamente un `ObjectId` de 12 bytes (timestamp + máquina/proceso + contador).

Un `_id` asignado manualmente debe ser único dentro de su colección: si repites uno, MongoDB rechaza el segundo documento con un error de clave duplicada.

Bloque 3 · Bases de datos y colecciones

5.1 Creación "perezosa"

MongoDB **no exige crear explícitamente** una base de datos ni una colección antes de usarla. `use nombre_bd` solo selecciona el contexto activo; la base de datos (y la colección) se crean de forma transparente en el momento en que se inserta el **primer documento**.

> **Atención.** Hasta que no insertes un documento, la base de datos **no aparecerá** en `show dbs`, aunque hayas ejecutado `use`.

5.2 Operaciones de administración sobre colecciones

Comando	Qué hace
db.createCollection("nombre")	Crea una colección de forma explícita (necesario para opciones especiales, p.ej. capped)
show collections	Lista las colecciones de la base de datos activa
db.coleccion.stats()	Muestra estadísticas (tamaño, nº documentos, índices...)
db.coleccion.renameCollection("nuevo")	Renombra la colección
db.coleccion.drop()	Elimina la colección por completo, con documentos e índices
db.dropDatabase()	Elimina la base de datos activa por completo

> **Atención.** `drop()` y `dropDatabase()` son **irreversibles**. Antes de ejecutarlas conviene comprobar con `db` que estás conectado a la base de datos correcta.

5.3 Namespace

Cada colección se identifica por su **namespace**: `nombre_de_la_base_de_datos.nombre_de_la_coleccion` (p. ej. `tienda_online.productos`). Debe ser único en todo el servidor, por lo que dos colecciones con el mismo nombre pueden coexistir si pertenecen a bases de datos distintas.

5.4 Esquema flexible: la trampa más habitual

Dentro de una misma colección, **no todos los documentos tienen que compartir los mismos campos**. MongoDB no lo impide por defecto. Esto es potente, pero exige disciplina: si unos documentos usan `categoria` y otros `categoría` (con tilde), las consultas devolverán resultados **incompletos sin ningún error**, porque para MongoDB son dos campos distintos.

> **Idea clave.** Mantener nombres de campo **consistentes** dentro de una misma colección es una buena práctica fundamental.

Bloque 4 · Operaciones CRUD

El capítulo más importante del manual: **todo** el resto del curso se apoya en estas cuatro operaciones.

Operación	Métodos	Equivalente SQL
Create	insertOne(), insertMany()	INSERT
Read	find(), findOne()	SELECT
Update	updateOne(), updateMany(), replaceOne()	UPDATE
Delete	deleteOne(), deleteMany()	DELETE

6.1 CREATE

```
db.productos.insertOne({ nombre: "Smartphone Pro", precio: 699.99 })
// -> { acknowledged: true, insertedId: ObjectId('...') }

db.productos.insertMany([ { ... }, { ... } ])  // array de documentos
```

6.2 READ

Operadores de comparación

Operador	Significa
$eq	Igual a
$ne	No igual
$gt	Mayor que
$gte	Mayor o igual
$lt	Menor que
$lte	Menor o igual
$in	Valor dentro de un array
$nin	Valor fuera de un array

Operadores lógicos

Operador	Significa
$and	Todas las condiciones (Y lógico)
$or	Al menos una condición (O lógico)
$not	Negación de una condición
$nor	Ni una ni la otra

Varias condiciones separadas por comas dentro de `{ }` ya se combinan de forma implícita con Y lógico: `$and` explícito solo hace falta para combinar varias condiciones **sobre el mismo campo**, o para mezclar `$and` con `$or`.

Proyección, sort(), limit(), skip()

```
db.productos.find({ categoria: "electronica" }, { nombre: 1, precio: 1, _id: 0 })
db.productos.find().sort({ precio: -1 }).limit(5)   // los 5 más caros
```

Idea clave. `sort()` sin `limit()` ordena toda la colección; `limit()` sin `sort()` devuelve "los N primeros que encuentre" **en orden no garantizado**.

6.3 UPDATE

Método	Qué hace
updateOne(filtro, cambios)	Actualiza el primer documento que cumple el filtro
updateMany(filtro, cambios)	Actualiza todos los documentos que cumplen el filtro
replaceOne(filtro, documento)	Sustituye el documento completo por uno nuevo

Atención. El **filtro** y los **cambios** son siempre dos argumentos distintos. Escribirlos como un único objeto, o usar `updateOne()` cuando el enunciado pide "todos los..." (que exige `updateMany()`), son los dos errores más habituales: MongoDB no lanza error, pero el resultado no es el esperado. Revisa siempre `matchedCount` y `modifiedCount` tras cada update.

Operador	Qué hace
$set	Establece el valor de un campo (lo crea si no existe)
$unset	Elimina un campo del documento
$inc	Incrementa o decrementa un valor numérico
$mul	Multiplica un valor numérico
$push	Añade un elemento a un array (permite duplicados)
$addToSet	Añade un elemento a un array solo si no existe ya
$pull	Elimina de un array los elementos que cumplan una condición

6.4 DELETE

Método	Qué hace
deleteOne(filtro)	Elimina el primer documento que cumple el filtro
deleteMany(filtro)	Elimina todos los documentos que cumplen el filtro

Atención. `deleteOne()`/`deleteMany()` reciben **un único argumento**: el filtro (a diferencia de update, que recibe dos). Escribir el filtro como **segundo** argumento —p. ej. `deleteMany({}, { categoria: "ropa" })`— hace que MongoDB lo ignore y borre siguiendo el primer argumento, que puede ser un filtro vacío `{}` que no excluye nada. Es **irreversible**. Norma de oro: antes de cualquier delete, ejecuta el mismo filtro con `find()` y comprueba que son exactamente los documentos que quieres borrar.

Bloque 5 · Índices y explain()

7.1 Qué es un índice

Un índice es una estructura adicional que acelera las consultas sobre un campo, a costa de espacio en disco y de mantenimiento en cada escritura. Sin índice sobre un campo, MongoDB hace un **escaneo completo** de la colección (COLLSCAN).

7.2 Índices como árboles B-Tree

- Búsqueda en un índice: complejidad aproximada **O(log n)**, frente a **O(n)** de un escaneo completo.
- Cada colección tiene **siempre** un índice por defecto y único sobre `_id`.

7.3 Límites de los índices

Límite	Valor
Número máximo de índices por colección	64
Campos máximos en un índice compuesto	32
Longitud máxima de una clave indexada	1024 bytes
Campos de tipo array por índice compuesto	1 (no se permite más de uno a la vez)

7.4 Tipos de índice

Tipo	Uso principal
Campo único	createIndex({ campo: 1 })
Compuesto	createIndex({ a: 1, b: -1 }) — el orden de los campos es crítico
Multiclave (array)	Indexa cada elemento del array por separado
Texto	$text / $search — máximo **1 índice de texto por colección**
TTL	expireAfterSeconds — elimina documentos automáticamente tras un tiempo
Parcial	partialFilterExpression — solo indexa el subconjunto que cumple un filtro
Único	{ unique: true } — falla si ya hay duplicados en ese campo

7.5 explain(): cómo verificar si una consulta usa índice

```
db.productos.find({ categoria: "electronica" }).explain('executionStats')
```

Modo	Qué hace
queryPlanner	Plan elegido, sin ejecutar la consulta (por defecto)
executionStats	Ejecuta la consulta y añade estadísticas reales
allPlansExecution	Como executionStats, pero con todos los planes alternativos

- `winningPlan.stage`: **COLLSCAN** = escaneo completo; **IXSCAN** = usó un índice.
- `totalDocsExamined` vs `nReturned`: si son iguales (y hubo IXSCAN), el índice es muy selectivo; si `totalDocsExamined` es mucho mayor, el índice ayuda poco.

Idea clave. Si ves `COLLSCAN` donde esperabas `IXSCAN`, falta un índice (o la consulta no lo está aprovechando por el orden de los campos).

7.6 La regla ESR para índices compuestos

Equality → Sort → Range (Igualdad, Orden, Rango): el orden recomendado para construir un índice compuesto eficaz.

- 1º los campos consultados por **igualdad** (==).
- 2º los campos usados para **ordenar** (sort).
- 3º los campos consultados por **rango** ($gt, $lt, $in...).

```
// Consulta: estado activo (igualdad), ordenado por apellido, edad > 30 (rango)
db.usuarios.createIndex({ estado: 1, apellido: 1, edad: 1 })
```

Atención. Un índice adicional `{estado: 1}` sería **redundante** si ya existe `{estado: 1, apellido: 1}`, porque las consultas por `estado` solo ya pueden usar el índice compuesto (es prefijo del otro).

Bloque 6 · Import/export y copias de seguridad

8. mongoimport / mongoexport

Se ejecutan desde la **terminal del sistema operativo**, nunca dentro del mongosh integrado en compass. Pensadas para mover datos puntuales (archivos JSON/CSV), no como estrategia de copia de seguridad: exportan una colección a la vez y no conservan los índices.

```
mongoimport --db=tienda_online --collection=productos --file=productos.json
mongoimport --db=tienda_online --collection=productos --jsonArray --
file=prod_array.json
mongoimport --db=tienda_online --collection=productos --type=csv --headerline --
file=productos.csv

mongoexport --db=tienda_online --collection=productos --out=productos_export.json
mongoexport --db=tienda_online --collection=productos --type=csv --
fields=nombre,precio --out=p.csv
```

Opción	Aplica a	Qué hace
--jsonArray	import/export	El archivo es un único array JSON, no un documento JSON por línea
--headerline	import (csv)	La primera línea del CSV son los nombres de campo
--type	import/export	Formato: json (por defecto) o csv
--fields	export (csv)	Obligatorio en CSV: columnas a exportar
--query	export	Filtro (misma sintaxis que find()) para exportar solo parte de la colección
--drop	import	Elimina la colección de destino antes de importar

Atención. Por defecto `mongoimport` espera **un documento JSON por línea** (NDJSON), no un array. Si tu archivo empieza por `[` y termina por `]`, necesitas `--jsonArray`. Y `mongoimport` trata **todos** los valores CSV como texto salvo que uses `--columnsHaveTypes`.

9. mongodump / mongorestore: copias de seguridad reales

A diferencia de mongoexport/mongoimport, trabajan con el formato binario **BSON** completo: preservan índices, tipos de datos exactos y pueden volcar varias colecciones a la vez. Son la herramienta correcta para una copia de seguridad fiel.

```
// Copia de una base de datos completa
mongodump --db=tienda_online --out=./backups/2026-10-07

// Restaurar (añade documentos; no sobrescribe por defecto)
mongorestore --db=tienda_online ./backups/2026-10-07/tienda_online

// Restauración LIMPIA: elimina antes la colección de destino
mongorestore --db=tienda_online --drop ./backups/2026-10-07/tienda_online
```

Atención. `--drop` elimina cada colección de destino **antes** de restaurar: es destructivo sobre los datos actuales. Comprueba siempre a qué servidor estás conectado antes de usarlo. Además, `--db`/`--collection` aparecen marcados como obsoletos al restaurar desde un directorio o archive; la alternativa recomendada es `--nsInclude="basedatos.*"`, aunque `--db`/`--collection` se siguen aceptando (y funcionan) hoy.

Idea clave. Buenas prácticas de backup: automatiza y programa las copias, guárdalas **fuera** del propio servidor, verifica periódicamente que la restauración funciona, y conserva varias versiones en el tiempo.

Bloque 7 · Chuleta de comandos

Resumen de una sola pasada de todos los comandos y operadores de la unidad, agrupados por bloque.

mongosh · navegación

Comando	Qué hace
show dbs	Lista bases de datos
use nombre_bd	Cambia/crea base de datos activa
db	Base de datos activa
show collections	Lista colecciones
db.coleccion.stats()	Estadísticas de la colección
db.coleccion.drop()	Elimina la colección
db.dropDatabase()	Elimina la base de datos activa

CRUD

Comando	Qué hace
insertOne(doc)	Inserta un documento
insertMany([docs])	Inserta varios documentos
find(filtro, proyección)	Consulta documentos
findOne(filtro)	Consulta un único documento
updateOne(filtro, cambios)	Actualiza el primero que cumple el filtro
updateMany(filtro, cambios)	Actualiza todos los que cumplen el filtro
replaceOne(filtro, doc)	Sustituye el documento completo
deleteOne(filtro)	Elimina el primero que cumple el filtro
deleteMany(filtro)	Elimina todos los que cumplen el filtro
.sort({campo:1|-1})	Ordena ascendente (1) o descendente (-1)
.limit(n) / .skip(n)	Limita resultados / salta los n primeros
.count() / .countDocuments()	Cuenta documentos

Operadores de consulta y actualización

Operador	Significa
$eq $ne $gt $gte $lt $lte	Comparación: igual, distinto, mayor(igual), menor(igual)
$in / $nin	Valor dentro / fuera de un array
$and $or $not $nor	Operadores lógicos
$set / $unset	Establece / elimina un campo
$inc / $mul	Incrementa / multiplica un valor numérico
$push / $addToSet / $pull	Añade (con/sin duplicados) / elimina de un array

Índices

Comando	Qué hace
createIndex({campo:1})	Índice de campo único (1 asc., -1 desc.)
createIndex({a:1,b:1})	Índice compuesto (orden ESR: igualdad → sort → rango)
createIndex({c:1},{unique:true})	Índice único
createIndex({c:'text'})	Índice de texto (máx. 1 por colección)
createIndex({f:1},{expireAfterSeconds:n})	Índice TTL
.explain('executionStats')	Analiza el plan de ejecución real (COLLSCAN vs IXSCAN)

Terminal del sistema operativo (fuera de mongosh)

Comando	Qué hace
mongoimport --db= --collection= --file=	Importa JSON/CSV a una colección
mongoexport --db= --collection= --out=	Exporta una colección a JSON/CSV
mongodump --db= --out=	Copia de seguridad binaria completa
mongorestore --db= --drop ruta/	Restaura una copia (--drop = restauración limpia)